

# Lab Report: Git & SSL Certificates

Technical Documentation | Linux & Version Control Systems

## 1. Introducción

Este documento detalla la resolución de un laboratorio técnico enfocado en la administración de seguridad mediante **OpenSSL** y la gestión de flujo de trabajo con **Git**. El objetivo es demostrar competencias en criptografía básica y control de versiones profesional.

### Key Skills Demonstrated:

- Generación de Llaves RSA y CSRs.
- Gestión de Certificados Autofirmados (Self-signed).
- Administración de Repositorios Git (Branching, Merging, Commits).
- Troubleshooting de permisos en sistemas Linux.

## 2. Seguridad y Certificados SSL

### Generación de CSR y Llave Privada Cifrada

Se generó una llave RSA de 4096 bits protegida con contraseña y su correspondiente solicitud de firma (CSR) en un solo paso técnico.

```
openssl req -new -newkey rsa:4096 -passout pass:kkcloud -keyout priv.key -out cert.csr
```

### Certificado Autofirmado (Self-Signed)

Para entornos de desarrollo, se creó un certificado válido por 365 días con un Common Name (CN) específico, omitiendo el cifrado de la llave para facilitar el arranque automático de servicios.

```
openssl req -x509 -newkey rsa:4096 -nodes -keyout priv.key -out kodecloud.crt -days 365
```

```
-subj "/CN=kodecloud"
```

## Inspección de Certificados

Uso de herramientas de auditoría para verificar la identidad de certificados existentes.

```
openssl x509 -in my.crt -noout -subject
```

## 3. Gestión de Control de Versiones con Git

### Flujo de Trabajo y Staging Selectivo

Se implementó una estrategia de *staging* selectivo para incluir únicamente archivos fuente C++ ( `*.cpp` ), manteniendo el repositorio limpio de archivos temporales.

```
cd /root/kode
git add *.cpp
git commit -m "Added C++ files"
```

### Estrategia de Ramificación (Branching)

Creación y gestión de líneas de tiempo paralelas para experimentación y aislamiento de funciones.

```
# Creación y cambio a rama testing
git checkout -b testing

# Eliminación segura tras volver a la rama principal
git checkout master
git branch -d testing
```

### Integración de Cambios (Merging)

Fusión de la rama de documentación al tronco principal (master) para consolidar el progreso del proyecto.

```
git checkout master
git merge documentation
```

## 4. Resolución de Problemas (Troubleshooting)

**Desafío:** Acceso denegado al intentar navegar por directorios protegidos con `sudo cd`.

**Solución:** Entender que `cd` es un comando interno del shell. Se utilizó `sudo -i` para obtener una shell de superusuario interactiva, permitiendo la gestión correcta del repositorio en `/root/`.

**Desafío:** Error `fatal: Unable to read current working directory`.

**Solución:** Refresco del entorno mediante la salida y re-entrada al directorio del proyecto (`cd ~ && cd /root/kode`), solucionando desincronizaciones del sistema de archivos virtual.

## 5. Conclusión

El laboratorio permitió validar el dominio de comandos críticos para la infraestructura moderna, integrando la seguridad desde el despliegue (SSL) hasta la trazabilidad del código (Git).